

CODE SPORT



In this month's column, we continue our discussion of the machine reading comprehension task.

Machine reading comprehension (MRC) task falls under the broader class of question-answering systems, as we discussed in last month's column. Given a passage of text and a set of questions, the task is to find the answers to the questions from the passage. In particular, we will focus on the simple problem of answer-extraction, where we assume that the answer to the question is present in the passage. Our task is to identify the span or the contiguous text locations, which contain the answer.

In the approach we discussed in last month's column, we proposed to create a fixed length representation of the passage P , a fixed length representation of the question Q , combine these two representations using an encoder, and then use the encoded representation as an input to a decoder to predict the answer span in the original passage. Can you identify the issues associated with this approach? While this approach is simple, the disadvantage is that the entire large passage of text gets compressed into a fixed-length vector representation.

For instance, let us assume that our passage consists of 100 sentences with each sentence containing approximately 30 words each. So the passage totally contains 3000 words. Let us assume that we have two questions, Q_1 and Q_2 , where the answer span for question Q_1 is $\langle 10, 15 \rangle$ and the answer span for Q_2 is $\langle 2900, 2910 \rangle$. Note that the answer span is specified in terms of the word indices of the passage of text. The sequence of this passage of text is converted into a fixed-length representation by passing it through a recurrent neural network. As standard recurrent neural networks have the exploding/vanishing gradient problem with long sequences of text, the standard practice is to use a gated variant of recurrent neural networks such as LSTMs or GRUs.

Let us assume that we use LSTM to encode this long passage of text, which contains around 3000 words. Even with LSTMs, we find that predicting pieces of information depending on individual words occurring much earlier in the sequence becomes difficult, as the information in the sequence gets compressed into a fixed-length vector. Given that we are compressing a 3000-word sequence into a fixed-length vector of the size $\langle 256 \rangle$, for instance, we are losing vital word level information, which could impact the capability to predict the answer span correctly. How do we then overcome the issue of the fixed-length vector representation of the passage? Instead of using a vector, can we use a matrix to encode the text sequence?

There are multiple ways in which we can create a matrix representation of the sequence of the text. Let us assume that as we pass the sequence of words in the passage of text through the LSTM, we extract the output of the LSTM after each time-step. Hence, we will have as many vector outputs as the number of the words in the text sequence. We can concatenate these vectors together to construct a matrix, which represents the passage of text. This can be further enhanced by passing the sequence of the passage of text through a bi-directional LSTM, and concatenating the forward and backward output vectors at each time-step to construct the encoded passage matrix.

By representing the passage as a matrix instead of a single vector, we manage to reduce the loss of information associated with a fixed length vector representation. Just as we represented the passage of text as a matrix, we can also represent the question text as a matrix. Given that we now have these two matrix representations, we can then combine them based on our choice of encoder, and feed the encoded representation to a decoder to predict the answer span.